

Karun Kumar Somu

Karunkumarsomu1228@gmail.com | + 1 (443) 998-6425

[Linkedin](#) | [GitHub](#) | [Portfolio](#)

Professional Summary:

- 7+ years of full-stack development experience delivering high-traffic, production-grade applications across Banking, Healthcare, Retail, and Insurance domains, with strong ownership across the entire development lifecycle from architecture design through deployment and post-production support.
- Hands-on experience building component-driven front-end applications with **ReactJS** and **TypeScript**, implementing unidirectional data flow using **Redux**, optimizing rendering performance with memoization and code-splitting, and ensuring accessibility and cross-browser compatibility.
- Deep expertise in **Java/J2EE** back-end engineering using **Spring Boot** and **Spring Cloud**, designing distributed systems with independently deployable microservices, inter-service communication via REST and messaging queues, and resilience patterns such as circuit breakers and retries.
- Experienced with the full **Spring** ecosystem including **Spring MVC**, **Spring Security**, **Spring Data JPA**, **Spring Batch**, **Spring AOP**, and **Spring Boot Actuator**, using these to build secure, observable, and maintainable enterprise applications.
- Strong command of **RESTful API** design principles including resource modeling, versioning, idempotency, and proper use of HTTP semantics, with hands-on experience designing **GraphQL** and event-driven APIs for high-scale enterprise systems.
- Proficient in **Core Java SE 8+** with practical application of the **Stream API**, **Lambda Expressions**, **Optional**, **CompletableFuture**, and the **Concurrency API** to write efficient, non-blocking backend logic in high-throughput systems.
- Strong database engineering background across relational (**PostgreSQL**, **Oracle**, **MySQL**) and NoSQL (**MongoDB**, **Cassandra**, **DynamoDB**) stores, with experience in schema design for both OLTP and analytical workloads, query optimization, and ORM configuration using **Hibernate**.
- Cloud-native development experience on **AWS**, designing serverless workflows with **Lambda**, managing containerized workloads on **EKS**, handling async messaging via **SQS**, and building observable systems using **CloudWatch** dashboards, alarms, and log insights.
- Experienced in building and operating **Apache Kafka**-based event streaming pipelines, including topic partitioning strategies, consumer group management, exactly-once semantics, and dead-letter queue handling for fault-tolerant data processing.
- Security-focused engineer with experience implementing **OAuth 2.0**, **JWT**, **RBAC**, and data encryption at rest and in transit using **AWS KMS**, with additional exposure to compliance requirements in regulated industries including PCI-DSS and HIPAA.

Technical Skills:

Languages	Java, Python, JavaScript, TypeScript, Kotlin, SQL
Java Technologies	JDBC, Servlets, JSP, JPA, JNDI, JMS, JavaBeans, EJB, Java Persistence API, Java Naming Directory
Java Frameworks	Spring, Spring Boot, Spring MVC, Spring Security, Spring Data JPA, Spring AOP, Spring Batch, Spring Cloud, Spring Boot Actuator, Hibernate, Log4J2
Web Technologies	HTML5, CSS3, JavaScript, AJAX, Bootstrap, JSON, XML, SASS, REST, SOAP, GraphQL, TypeScript

Web Frameworks	Angular, ReactJS, Node.js, Vue.js, Next.js, Redux, React Query, React Hook Form, Express.js
Cloud	AWS (EC2, S3, RDS, EKS, ECS, Lambda, SQS, SNS, CloudWatch, CodePipeline, Glue, KMS, X-Ray), GCP, Azure
Databases	PostgreSQL, MySQL, Oracle, MongoDB, Cassandra, DynamoDB, Redis, AWS Redshift
CI/CD Tools	Jenkins, GitLab CI, AWS CodePipeline, GitHub Actions, SonarQube, Maven, Gradle
Containerization	Docker, Kubernetes (EKS, GKE), Helm, Terraform, AWS ECS
Messaging	Apache Kafka, RabbitMQ, AWS SQS, AWS SNS, ActiveMQ
Testing	JUnit, Jest, Mockito, Selenium, RestAssured, Pact, Gatling, TestNG
OS & Tools	Linux, Windows, UNIX, Git, SVN, Splunk, Datadog, Swagger, OpenAPI, Postman

Professional Experience:

Client: Volkswagen, Rochester, MI, USA

October 2024 – Present

Role: Sr. Full Stack Developer

Project Description: Worked on Volkswagen's Connected Vehicle and Dealer Management Platform — a next-generation digital ecosystem enabling vehicle configuration, dealer inventory tracking, service scheduling, and real-time vehicle telemetry dashboards. The platform supports millions of users across North America and required high availability, secure data handling, and a responsive UI across web and mobile devices.

Responsibilities:

- Built a reusable React component library using **TypeScript** and **Storybook**, standardizing UI patterns across 12 teams and significantly reducing redundant front-end component implementations across product squads.
- Collaborated closely with product managers and UX designers to translate complex vehicle workflows into intuitive **React** interfaces, conducting technical feasibility reviews during sprint planning to prevent scope creep.
- Implemented client-side form validation and multi-step dealer configuration wizards using **React Hook Form** and **Redux Toolkit**, improving data entry accuracy for vehicle configuration and service scheduling flows.
- Designed the microservices decomposition strategy for the platform, breaking a legacy monolith into domain-aligned **Spring Boot** services with clearly defined contracts, enabling independent deployability and team ownership per business capability.
- Engineered a real-time vehicle telemetry notification system using **Apache Kafka** with partitioned topics per vehicle type, ensuring ordered delivery and exactly-once processing semantics for high-frequency telematics events.
- Introduced **Spring Boot Actuator** health endpoints and custom metrics exposed to **AWS CloudWatch**, enabling proactive alerting on error rate spikes and memory pressure before they impacted end users.
- Designed and enforced API versioning strategies using URL-based versioning and **OpenAPI** specifications, ensuring backward compatibility for all dealer integrations during iterative service upgrades.
- Implemented a token refresh strategy using **OAuth 2.0** and **JWT** with sliding session windows and silent re-authentication flows, improving user experience without compromising the security posture.
- Implemented **Spring Security** at the API gateway level with fine-grained **RBAC** policies, restricting access to sensitive endpoints based on user roles and entitlements defined in the identity provider.
- Optimized critical **PostgreSQL** queries for dealer inventory lookups and vehicle history pagination, reducing p99 API response times from 850ms to under 120ms through composite indexing and query plan analysis.

- Configured **AWS RDS** Multi-AZ deployments with automated failover and read replicas, ensuring database high availability and offloading reporting workloads from the primary transaction database.
- Worked with the data engineering team to design **AWS Glue** pipelines that extracted vehicle data from **PostgreSQL** into **S3**-backed data lake partitions, enabling downstream analytics and regulatory reporting workflows.
- Led the migration from EC2-based deployments to **AWS EKS** using Helm charts, configuring pod autoscaling rules tied to Kafka consumer lag metrics to dynamically scale processing capacity under peak load.
- Drove adoption of **blue-green deployments** across all platform services, eliminating deployment-related outages and enabling same-day rollbacks when post-release anomalies were detected in production monitoring.
- Containerized all services using **Docker** with multi-stage builds, reducing final image sizes by 60% and improving deployment pipeline execution time through optimized layer caching strategies.
- Established branch protection rules and enforced mandatory **SonarQube** quality gates in **Jenkins** pipelines, significantly reducing the production defect escape rate through automated static analysis on every pull request.
- Integrated **Splunk** dashboards with structured JSON logs emitted by all microservices, enabling the operations team to trace individual transactions end-to-end across service boundaries within minutes.

Environment: Java, Spring Boot, Spring Cloud, Spring Security, React, TypeScript, Redux Toolkit, PostgreSQL, Apache Kafka, AWS EKS, AWS RDS, AWS CloudWatch, Docker, Jenkins, OAuth 2.0, OpenAPI, JUnit, Mockito, Splunk, SonarQube, Helm, Git.

Client: Delta Dental Ins, San Francisco, CA, USA

November 2022 – September 2024

Role: Full Stack Developer

Project Description: Developed and enhanced Delta Dental's Member and Provider Portal — a dental insurance management platform enabling members to view coverage details, manage claims, locate in-network providers, and access treatment history. The platform integrated with multiple dental data sources and insurance systems through APIs and required strict HIPAA compliance.

Responsibilities:

- Built a provider search feature in **ReactJS** with debounced API calls, skeleton loading states, and client-side result caching using **React Query**, cutting average search interaction time significantly compared to the previous implementation.
- Developed a dynamic benefits display module in **React** that rendered plan-specific benefit structures from a configuration-driven API, eliminating the need for hardcoded UI logic per insurance plan type.
- Implemented accessible, WCAG-compliant form components for the telehealth scheduling workflow using **React Aria** and semantic HTML, ensuring usability for members relying on assistive technologies.
- Architected the claims data access layer using **Spring Data JPA** with dedicated read replicas for reporting queries and write-optimized paths for claims submission, reducing database contention and improving throughput under concurrent load.
- Optimized **Spring Boot** application startup time by 40% through lazy bean initialization, conditional auto-configuration exclusions, and profile-based component loading tailored for containerized deployment environments.
- Built reusable **Spring Batch** job templates for nightly eligibility file processing, supporting configurable chunk sizes, skip policies, and retry limits that handled multi-million record files reliably without memory overflow.
- Integrated **HL7 FHIR R4** APIs to enable standardized patient record exchange with partner EHR systems, implementing FHIR resource mapping, validation, and error handling to ensure data fidelity across organizational boundaries.
- Enforced **API Gateway** rate limiting and request throttling policies to protect backend services from traffic spikes during open enrollment periods when member portal usage increased by 300%.

- Conducted threat modeling sessions with the security team, identifying and remediating high-severity vulnerabilities including insecure direct object references and missing transport layer security in third-party API integrations.
- Modeled **Cassandra** tables using query-first design principles, creating partition key strategies that evenly distributed member data across nodes and delivered sub-10ms response times for eligibility lookups at scale.
- Designed a HIPAA-compliant audit trail system capturing every PHI access event with user identity, timestamp, and data classification, storing encrypted audit logs in **AWS S3** with lifecycle policies enforcing 7-year retention.
- Implemented field-level encryption for sensitive member attributes using **AWS KMS** customer-managed keys, ensuring PHI remained protected even in the event of unauthorized database access.
- Replaced a synchronous claims adjudication process with an event-driven pipeline using **Apache Kafka** and **AWS Lambda**, decoupling intake from processing and significantly reducing average claim resolution time.
- Configured **AWS SQS** dead-letter queues with **CloudWatch** alarms for failed claim processing events, enabling the operations team to investigate and reprocess failures without data loss or manual coordination.
- Managed **Kubernetes** namespace isolation between development, staging, and production environments, enforcing network policies and resource quotas that prevented cross-environment data leakage and resource contention.
- Implemented distributed tracing using **AWS X-Ray** across all Lambda functions and API Gateway endpoints, providing end-to-end visibility into claims processing latency and identifying bottlenecks in third-party EHR integrations.
- Wrote comprehensive end-to-end test suites using **Selenium** and **RestAssured**, integrated into the **CI/CD** pipeline to automatically catch UI and API regressions before every staging deployment.

Environment: Java, Spring Boot, Spring Data JPA, Spring Batch, React, ReactQuery, Cassandra, Apache Kafka, AWS Lambda, AWS SQS, AWS S3, AWS KMS, AWS X-Ray, Docker, Kubernetes, HL7 FHIR R4, Jenkins, JUnit, RestAssured, Selenium, Git.

Client: Ulta Beauty, Bolingbrook, IL, USA

January 2021 – October 2022

Role: Software Developer

Project Description: Contributed to Ulta Beauty's e-Commerce Platform Modernization initiative, migrating legacy monolithic services to a cloud-native microservices architecture covering product catalog, inventory management, order processing, and personalization engine services, supporting Ulta's omni-channel retail strategy for both online and in-store beauty shopping experiences.

Responsibilities:

- Developed a multi-step checkout flow in **ReactJS** with real-time inventory validation, address autocomplete, and optimistic UI updates, delivering a seamless purchase experience that reduced cart abandonment during checkout.
- Implemented server-side rendering with **Next.js** for product listing and detail pages, improving Largest Contentful Paint (LCP) scores by 45% and enabling proper crawlability of dynamically generated content for SEO.
- Built a centralized error boundary and logging strategy in **React** using error boundary components and window.onerror handlers that captured client-side exceptions and forwarded structured error payloads to **Datadog** for frontend observability.
- Led the migration of the product catalog service from a legacy Oracle-backed monolith to a **Spring Boot** microservice backed by **MongoDB**, reducing average product search latency by 70% through denormalized document design and full-text search indexing.
- Utilized **Java 8 Stream API** and **Lambda Expressions** to refactor legacy imperative collection processing into declarative pipelines, improving code readability and reducing cyclomatic complexity across service classes.

- Built a real-time inventory sync pipeline using **Apache Kafka**, streaming warehouse stock updates to the storefront with end-to-end latency under 500ms, eliminating overselling incidents that had previously caused customer-facing errors.
- Implemented **GraphQL** federation across the product catalog and inventory services using Apollo Gateway, allowing the storefront to fetch composite product data in a single network request instead of multiple sequential REST calls.
- Designed consumer-driven contract tests using **Pact** between the storefront and catalog/inventory services, preventing breaking API changes from reaching production and enabling confident independent service deployments.
- Collaborated with the mobile team to design a shared **REST API** contract that served both web and mobile clients, using content negotiation and response shaping to minimize over-fetching on bandwidth-constrained mobile connections.
- Implemented a personalization engine integration that consumed **Kafka** events from the browsing activity stream and updated product recommendation rankings in **DynamoDB**, enabling real-time personalized experiences without blocking the page render path.
- Worked with the data science team to instrument product interaction events using a custom **Kafka**-based analytics pipeline, feeding clickstream data into **AWS Redshift** for A/B test analysis and conversion funnel reporting.
- Configured **AWS CloudFront** distributions with cache behaviors tuned per content type, achieving a 92% CDN cache hit ratio and significantly reducing origin load on **AWS S3** during peak traffic and promotional events.
- Defined **Terraform** modules for provisioning **AWS ECS** task definitions and ALB listener rules, enabling infrastructure-as-code practices and repeatable environment provisioning for new microservices.
- Participated in bi-weekly architecture guild meetings, presenting proposals for shared platform capabilities including a centralized feature flag service built on **AWS AppConfig** adopted by 5 product teams.
- Maintained and improved **Docker** base images used across 8 microservices, enforcing security scanning with Trivy in **GitLab CI** to identify and patch known CVEs before container images were promoted to production registries.
- Integrated automated performance tests into **GitLab CI** using Gatling load scripts, catching throughput regressions before production release and establishing baseline performance SLAs for all critical storefront APIs.
- Wrote detailed **JUnit** and **Mockito** unit tests with parameterized test cases covering edge conditions in pricing, discount, and inventory reservation logic, maintaining over 88% line coverage across all service modules.

Environment: Java 8, Spring Boot, ReactJS, Next.js, GraphQL, Apollo Gateway, MongoDB, DynamoDB, Apache Kafka, AWS S3, AWS CloudFront, AWS ECS, AWS Redshift, AWS AppConfig, Terraform, Docker, GitLab CI, JUnit, Mockito, Pact, Gatling, Datadog, Git.

Client: Iprism Technologies, Hyderabad, India

July 2019 – December 2020

Role: Java Developer

Project Description: Worked on an enterprise Insurance Policy Administration System at Iprism Technologies serving US-based insurance clients. The system handled policy issuance, endorsements, renewals, claims intake, and agent portal workflows. The application was built on a J2EE stack with ongoing modernization efforts toward RESTful services and Angular-based frontends.

Responsibilities:

- Built an **Angular** agent portal with reactive forms, real-time field-level validation, and dynamic form sections that rendered conditionally based on selected policy type, reducing agent data entry errors during UAT testing.

- Developed reusable **JSP** tag libraries and layout templates for the agent-facing web application, ensuring consistent UI rendering across all policy workflow screens and reducing front-end duplication across the portal.
- Developed the premium calculation engine using **Java EE** and **Spring MVC**, implementing a rule-based pricing model that evaluated multiple underwriting factors and reduced quote generation time significantly through in-memory caching with **Ehcache**.
- Implemented **Spring Batch** partitioned jobs for parallel processing of policy renewal notifications, with configurable chunk sizes, skip limits, and retry policies that ensured zero data loss even when downstream mail service failures occurred.
- Applied **SOLID** design principles and **Gang of Four** patterns including Strategy, Factory, and Decorator across the underwriting rules engine, making it straightforward to add new coverage types without modifying existing rule evaluators.
- Built a document generation module using **Apache POI** and **iText** that produced policy declaration pages and renewal notices in PDF and Word formats, with dynamic data binding from the policy management database.
- Replaced legacy SOAP-based integration endpoints with versioned **RESTful APIs** using **Spring REST**, decoupling the policy system from tightly bound legacy billing and claims platforms and reducing average integration response time significantly.
- Configured **Spring Security** with LDAP-based authentication for the agent portal, implementing role-based access control that restricted policy endorsement and cancellation actions to appropriately credentialed agent profiles.
- Designed and tuned **Oracle** stored procedures for policy renewal batch processing, implementing bulk collect and FORALL constructs that dramatically reduced nightly renewal job execution time compared to row-by-row processing.
- Worked with the database team to analyze slow query logs in **Oracle**, adding targeted indexes on high-frequency policy lookup columns that reduced average claim intake query execution time from 4 seconds to under 300ms.
- Implemented **Hibernate** second-level caching using **Ehcache** for frequently accessed reference data such as coverage codes and state regulations, reducing redundant database reads during peak policy issuance windows.
- Introduced structured logging using **Log4J2** with JSON-formatted output and correlation IDs across all service calls, enabling the operations team to trace policy processing failures end-to-end across service boundaries for the first time.
- Maintained **Maven** multi-module project structure, managing dependency version alignment across 6 sub-modules and resolving transitive dependency conflicts that had previously caused intermittent build failures in the CI environment.
- Actively contributed to sprint retrospectives by identifying recurring build and deployment friction points, leading to the introduction of a shared **Jenkins** pipeline template that standardized CI processes across all project modules.
- Wrote comprehensive **JUnit** test suites covering all premium calculation rules and edge cases, achieving 88% line coverage and enabling the team to confidently refactor the pricing engine without introducing regression defects.

Environment: Java, Spring MVC, Spring Batch, Spring Security, Hibernate, Ehcache, Angular, Oracle, PL/SQL, Spring REST, Log4J2, Apache POI, iText, JSP, JUnit, Maven, SVN, WebLogic, Jenkins.

Education:

Master of Science in Computer Science, University of Maryland, Baltimore County, MD, USA

Bachelor of Engineering in Computer Science, Sathyabama Institute of Science and Technology, Chennai, India